



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

agentní inženýrství, řídicí harness, GitHub App, DarkFactory, automatizace vývoje, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

agentic engineering, harness architecture, GitHub App, DarkFactory, software development automation, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace a vymezení problému	7
1.2 Cíl práce a výzkumné otázky	7
1.3 Metodika práce	8
2 Teoretická část	9
2.1 Správa verzí (Git a GitHub)	9
2.1.1 Úvod do správy verzí a GitHubu	9
2.1.2 Větve (Branches) a izolace kódu	9
2.1.3 Model pull requestu (PR)	9
2.1.4 Slučování změn (Squash and Merge)	10
2.1.5 Kontinuální integrace (CI a GitHub Actions)	10
2.1.6 Požadované kontroly (Required Checks) a ochrana větví	11
2.2 Kognitivní jádro a správa kontextového okna	11
2.2.1 Úvod do velkých jazykových modelů	12
2.2.2 Tokeny, tokenizace a embedding	12
2.2.3 Tahy a správa KV cache	12
2.2.4 Kompakce kontextu a ztrátová komprese	12
2.2.5 Sémantický posun (Semantic Drift)	13
2.2.6 Alternativní paměťové architektury (RAG a stavový graf)	13
2.2.7 Degradace pozornosti (Context Rot)	13
2.2.8 Promptové inženýrství a negativní instrukce	13
2.3 Architektura řídicího harnessu a orchestrace	14
2.3.1 Úvod do řídicích harnessů	14
2.3.2 Autonomní agent vs. konverzační chatbot	14
2.3.3 Prováděcí cyklus ReAct	14
2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)	15
2.3.5 Patologie divergence: perseverace a oscilace	15
2.3.6 Dovednosti (Skills)	16
2.3.7 Model Context Protocol (MCP servery)	16

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow	16
2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)	16
3 Praktická část	18
3.1 Cíl a rozsah systému DarkFactory	18
3.2 Architektura a životní cyklus požadavku	18
3.3 Orchestrační jádro a validační infrastruktura	19
3.4 Systém revizních značek pro lidský dohled nad akademickým textem	20
4 Výsledky a diskuse	22
4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami	22
4.2 Provozní metriky a spolehlivost agentních běhů	23
4.3 Poznatky a systémová úskalí z provozu	23
4.4 Diskuse: Porovnání architektur	23
4.5 Systémová a metodická omezení	24
5 Závěr	25
Seznam příloh	26
A Obsah přiloženého média	27
B Schéma konfiguračního manifestu darkfactory.json	28
C Sdílené workflow pro GitHub Actions	29
D Systémové prompty plánovacího a kódovacího agenta	30
E Protokol revizních značek v sazebním systému Typst	31
Seznam zdrojů	33

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	15
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	19

KONCEPT

1 Úvod

1.1 Motivace a vymezení problému

- **Současný stav automatizace:** Sestavení programu, běh testů, statická analýza i nasazení do produkce jsou dnes plně automatizovány stroji. Úzké hrdlo procesu tvoří samotná modifikace zdrojového kódu — časová prodleva mezi zadáním požadavku a vytvořením otestované změny.
- **Koncept temné továrny (*Dark Factory*):** Průmyslový model plně automatizovaného provozu běžícího bez stálé lidské obsluhy (není nutné svítit). Cílem není vytlačení člověka (inženýr a konstruktér určují záměr a specifikaci), nýbrž úplná eliminace člověka z mechanických, opakujících se úkonů.
- **Limity stávajících nástrojů na bázi LLM:** Většina stávajících řešení (chatovací asistenti, doplňování kódu) řeší pouze izolovaný krok — návrh fragmentu kódu. Chybí ucelená integrace do vývojového cyklu: kontextové zasazení, exekuce v repozitáři, řešení syntaktických regresí a řízení deterministických záruk.
- **Ústřední inženýrská otázka:** Nezkoumáme triviální otázku, zda jazykový model dokáže napsat fragment kódu, nýbrž jaká kontrolní architektura (**harness**) musí model obklopovat, aby jeho výstupům bylo možné v produkčním repozitáři spolehlivě důvěřovat.

 **Návrh na vylepšení: Doporučení k motivaci:** Doporučujeme doplnit srovnávací diagram analogie mezi fyzickou temnou továrnou (materiál → montážní linka → kontrola jakosti → expedice) a agentním vývojem softwaru (GitHub Issue → agentní harness → CI validace → PR/Release). Vizuální metafora výrazně zlepší srozumitelnost pro komisi.

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídicího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíkových rozhodovacích bodech.

Dílčí cíle:

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

Výzkumné otázky:

- **VO1 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po pull request strukturovat tak, aby role vývojáře spočívala výhradně v architektonickém doзору a schvalování záměru (Human Gate), bez nutnosti ručního psaní rutinního kódu?
- **VO2 (Řízení divergence a spolehlivost smyčky):** Jakými architektonickými mechanismy lze v harnessu spolehlivě zabránit patologiím modelu (perseveraci, oscilaci a zacyklení v ReAct smyčce)?
- **VO3 (Integrita paměti a eliminace sémantického posunu):** Jak spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a ztrátě architektonických invariantů při kompresi?

 **Hlubková kritika / Oponentura: Oponentura k výzkumným otázkám:** Otázka VO1 je formulována binárně („Lze vývojový proces strukturovat...“), což svádí k tautologické odpovědi. Rigorózní oponent bude žádat empirické vymezení: Jaké procento rutinních úloh (např. oprava chyby se selhávajícím testem vs. komplexní refaktoring) harness reálně odbaví bez ručního zásahu do kódu? Doporučujeme otázku v obhajobě doplnit o kritérium mezní složitosti úkolu a míry redukce kognitivní zátěže člověka.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Postup sleduje strukturu inženýrského cyklu:

- **1. Konceptní analýza:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Architektonický návrh:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

 **Strukturální upozornění: Chybějící evaluační rámec v metodice:** Metodika práce v současné podobě popisuje inženýrský postup, ale postrádá formální specifikaci evaluačního rámce: definici vzorku úloh pro ověření spolehlivosti (syntetické úlohy vs. reálné bugfixy), stanovení kontrolních metrik (úspěšnost na první pokus, spotřeba tokenů na úspěšný PR) a srovnávací baseline.

2 Teoretická část

2.1 Správa verzí (Git a GitHub)

2.1.1 Úvod do správy verzí a GitHubu

- **Proč správa verzí:** Jazykové modely generují kód na základě pravděpodobnosti a dělají chyby. Správa verzí poskytuje bezpečné prostředí, kde lze každou změnu sledovat, testovat a v případě chyby kdykoliv vrátit k funkčnímu stavu.
- **Nástroj Git (1):** Standardní distribuovaný verzovací nástroj. Kód se ukládá do historie v podobě jednotlivých revizí (*commitů*). Agent i vývojář pracují lokálně s plnou kopií repozitáře a mohou provádět úpravy, větvení i testování nezávisle na síti.
- **Platforma GitHub:** Webová služba postavená nad Gitem, která slouží jako centrální bod pro sdílení kódu a automatizaci:
 - **Zadávání a sledování úkolů (Issues):** Textové zadání požadavků, hlášení chyb a diskuse, ze kterých agent čerpá zadání.
 - **Přehled a kontrola změn (Pull Requests):** Rozhraní pro revizi diffu a schvalování kódu před jeho začleněním.
 - **Automatizace (GitHub Actions):** Běhové prostředí pro automatické spouštění testů, linterů a překladů.
- **Praktická role v práci:** Namísto teoretických abstrakcí práce přímo využívá Git a ekosystém GitHubu jako reálný základ pro řízení autonomního vývoje.

2.1.2 Větev (Branches) a izolace kódu

- **Větev (Branches):** Samostatné vývojové linky v Gitu odbočené ze základního kódu. Umožňují pracovat na novém úkolu odděleně od ostatních.
- **Hlavní větev (`main`):** Reprezentuje stabilní, otestovaný stav projektu připravený k nasazení. Do této větve nikdo (ani člověk, ani agent) nezapíše přímo.
- **Pracovní větev agenta:** Agent si pro každý úkol vytvoří novou samostatnou větev (např. `task/...` nebo `agent/...`):
 - **Oddělení chyb a pokusů:** Pokusy, mezistavy ani nefunkční kód neovlivňují stabilitu hlavní větve ani práci ostatních vývojářů.
 - **Bezpečné zahození:** Pokud se agent vydá špatným směrem nebo selže, celou větev lze smazat jedním příkazem bez následků pro projekt.
- **Aktualizace větve:** Pokud se hlavní větev během práce posune dopředu, pracovní větev agenta se zaktualizuje (`rebase` nebo `merge`), aby se předešlo konfliktům při slučování.

2.1.3 Model pull requestu (PR)

- **Pull Request (PR):** Standardní způsob, jak na GitHubu navrhnout změny z pracovní větve k začlenění do větve hlavní (`main`).

- **Komponenty rozhraní PR:**
 - **Rozdíl kódu (*Diff*):** Přehledné řádkové srovnání — zeleně přidané řádky, červeně odebrané řádky.
 - **Popis změn:** Agent v popisu PR srozumitelně shrne, jaké změny provedl, proč je zvolil a na jaké issue reagoval.
 - **Výsledky kontrol:** Přehled stavu automatických testů z GitHub Actions (zelená / červená).
 - **Diskusní vlákno:** Prostor pro komentáře, připomínky a požadavky na úpravy ze strany vývojáře.
- **Schvalovací brána člověka (*Human Gate*):** PR slouží jako hlavní kontrolní bod podle principu *Human-in-the-loop*. Člověk zkontroluje navržený kód a rozhodne o jeho schválení či zamítnutí.

2.1.4 Slučování změn (*Squash and Merge*)

- **Způsoby sloučení na GitHubu:**
 - **Klasický merge commit:** Přenese všechny jednotlivé commity z větve a vytvoří slučovací uzel.
 - **Rebase and Merge:** Přeskládá commity z větve lineárně za sebou.
 - **Squash and Merge:** Vezme všechny commity z pracovní větve, spojí je do jediného nového commitu a ten vloží do `main`.
- **Význam Squash and Merge pro agenty:**
 - **Skrytí interního šumu:** Agent při řešení úlohy často vytvoří desítky drobných commitů (opravy překlepů, dílčí pokusy po selhání testu). Tyto mezikroky nemají pro historii projektu trvalou hodnotu.
 - **Čistá a přehledná historie:** V hlavní větvi repozitáře zůstane za každý vyřešený úkol právě jeden ucelený commit s popisem.
 - **Jednoduchý návrat změn (`git revert`):** Pokud by změna v budoucnu způsobila problém, lze celý úkol vrátit jediným příkazem bez nutnosti rozplétat dílčí mezikroky.

2.1.5 Kontinuální integrace (CI a GitHub Actions)

- **Kontinuální integrace (CI) (2):** Automatizované sestavování a testování kódu při každé změně (pushnutí do větve nebo otevření pull requestu).
- **GitHub Actions:** Nástroj přímo integrovaný v GitHubu, který spouští definované pracovní postupy (*workflows*) v izolovaných virtuálních prostředích (např. kontejnerech).
- **Ověření funkčnosti kódu:** Samotný jazykový model kód pouze generuje na základě pravděpodobnosti; neví, zda je kód funkční. Skutečné ověření probíhá až v CI spuštěním překladače a testů.

- **Zpětná vazba pro agenta:** Pokud krok v CI selže, chybový protokol slouží agentovi jako přesný vstup pro další iteraci opravy.
- **Nezávislost na lokálním prostředí:** CI běží na čistém systému se stanovenými verzemi závislostí, což vylučuje chyby způsobené odlišnostmi v lokálním nastavení vývojáře.

 **Hlubková kritika / Oponentura: Nestálost testů (Flaky Tests) v integračních bězích:** Spoléhání se na automatické testy v CI naráží na problém nestálých testů (*flaky tests*), které občas selžou kvůli časování, síťové odezvě či asynchronním stavům, aniž by kód obsahoval chybu. Pokud agent narazí na takto náhodně selhávající test, může začít nesmyslně upravovat správný kód ve snaze chybu odstranit. CI pipeline proto musí nestálé testy minimalizovat nebo umožnit automatické opakování selhaného běhu v čistém prostředí.

2.1.6 Požadované kontroly (Required Checks) a ochrana větví

- **Pravidla ochrany větví (Branch Protection Rules):** Bezpečnostní nastavení GitHubu chrání větev `main` před nechtěným poškozením:
 - ▶ Zákaz přímého pushování do hlavní větve.
 - ▶ Zákaz mazání hlavní větve a přepisování její historie (`force push`).
- **Požadované kontroly (Required Checks):** Seznam úloh v GitHub Actions, které musí projít úspěšně (zelený stav), aby bylo technicky možné PR sloučit:
 - ▶ **Linter a formátování:** Kontrola dodržení kódového stylu a základních syntaktických pravidel.
 - ▶ **Typová kontrola a build:** Ověření, že kód lze bez chyb zkompileovat a typy odpovídají.
 - ▶ **Automatické testy:** Běh jednotkových a integračních testů s definovaným očekávaným chováním.
- **Pravidlo deterministického výsledku:** Každá kontrola musí skončit jednoznačným stavem (úspěch / selhání). Tiché přeskočení testu nesmí být považováno za splněnou podmínku.
- **Povinné schválení člověkem:** Požadavek na explicitní schválení kódu lidským vývojářem před sloučením do produkční větve.

2.2 Kognitivní jádro a správa kontextového okna

 **Metodické vymezení / Rozsah práce: Metodické vymezení a rozsah práce:** Těžištěm této práce není strojové učení, matematická optimalizace vah ani trénování neuronových sítí. Jazykový model vnímáme jako hotovou inferenční komponentu vystupující v roli stochastického kognitivního jádra. Ústředním předmětem zkoumání je **agentní inženýrství** (*agentic engineering*) a **architektura řídicího harnessu** pro autonomní vývoj softwaru. Následující text je proto záměrně zredukován na nezbytné konceptuální minimum potřebné pro pochopení kontextového okna, spotřeby tokenů, degradace pozornosti a rozhraní nástrojů.

2.2.1 Úvod do velkých jazykových modelů

- **Velké jazykové modely (LLM) (3):** Architektura dekodérového transformeru (*Decoder-only*, např. řady Claude, GPT, LLaMA, DeepSeek).
- **Stochastický autoregresivní generátor:** Model na základě zadané textové historie (kontextu) opakovaně predikuje nejpravděpodobnější následující symboly.
- **Role v agentním inženýrství:** Model nevystupuje jako orákulum se znalostí reálného světa, nýbrž jako generátor hypotéz, kódu a strukturovaných volání nástrojů řízený promptem a kontextem.

2.2.2 Tokeny, tokenizace a embedding

- **Tokeny:** Základní diskrétní celočíselné jednotky, se kterými neuronová síť počítá (podmnožiny slov, slabiky, znaky).
- **Tokenizér:** Deterministický algoritmus (např. Byte-Pair Encoding, BPE) převádějící vstupní text na posloupnost tokenů a naopak.
- **Embedding:** Projekce tokenů do vícerozměrného vektorového prostoru, v němž geometrická vzdálenost odpovídá sémantické příbuznosti pojmů.
- **Jazyková asymetrie tokenizace:** Text v češtině spotřebovává kvůli bohaté flexi a diakritice 2× až 3× více tokenů než ekvivalent v angličtině.
 - ▶ **Inženýrské doporučení:** Vnitřní systémové prompty, technické plány i logy harnessu vést v angličtině pro minimalizaci nákladů a latence.

2.2.3 Tahy a správa KV cache

- **Tahy (Turns):** Diskrétní kroky výměny informací v agentní smyčce:
 - ▶ **Tah uživatele / prostředí (User/Environment Turn):** Zadání úkolu nebo vnější událost.
 - ▶ **Tah modelu (Model Turn):** Odpověď modelu nebo emitování požadavku na nástroj.
 - ▶ **Tah vykonání nástroje (Tool Execution Turn):** Výsledek operace vrácený harnessu.
- **KV cache (Key-Value Cache):** Ukládání mezivýpočtů matic pozornosti klíčů a hodnot pro již zpracovanou historii tahů.
 - ▶ **Výpočetní úspora:** Inferenční engine při novém tahu nemusí přepočítávat celou historii od začátku.
- **Kontextové okno (Context Window):** Pevně limitovaná kapacita paměti modelu. Omezeno kvadratickou složitostí mechanismu pozornosti ($O(N^2)$ vzhledem k délce sekvence N) a velikostí paměti GPU akceleratorů.

2.2.4 Kompakce kontextu a ztrátová komprese

- **Kompakce historie (Compaction):** Vyzvání modelu k vytvoření syntetického souhrnu dosavadního průběhu sezení pro uvolnění kapacity kontextového okna.

- **Destruktivní ztrátová komprese:** Autoregresivní model při rekurzivním zkracování podléhá konfirmačnímu zkreslení (*confirmation bias*) a preferuje fakta odpovídající jeho vnitřním statistickým asociacím:
 - **Vymazání deterministických detailů:** Nevratný zánik čísel řádků, signatur privátních funkcí, přesných cest v souborech a doslovných chybových hlášení kompilátoru.
 - **Ztráta negativních omezení:** Zákazy (neměnit veřejné API, nepřidávat externí knihovny) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.

2.2.5 Sémantický posun (Semantic Drift)

- **Mechanismus posunu:** Kaskádové kumulování drobných zkreslení a halucinací při vícenásobné rekurzivní kompresi ($S_{k+1} = f(S_k, \Delta_k)$).
- **Efekt tiché pošty:** Drobné nepřesnosti z kola k jsou v kole $k + 1$ přijaty jako nezvratná historická fakta.
- **Důsledek pro repozitář:** Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému.

2.2.6 Alternativní paměťové architektury (RAG a stavový graf)

- **Hierarchická epizodická paměť (RAG):** Ukládání doslovné historie tahů a výpisů nástrojů do externí databáze; selektivní injekce pouze bezprostředně relevantních fragmentů do aktivního okna.
- **Persistentní graf stavu projektu (Project State Graph):** Udržování explicitního strukturovaného stavu repozitáře (změněné soubory, otevřené úkoly, výsledky testů, invarianty) mimo kontextové okno.

2.2.7 Degradace pozornosti (Context Rot)

- **Context Rot:** Degradace schopnosti modelu rovnoměrně využívat informace v dlouhém kontextovém okně (jev *Lost in the Middle* (4)).
- **Multi-Needle Reasoning:** Schopnost současně nalézt a logicky propojit několik na sobě závislých faktů napříč soubory; s rostoucí délkou kontextu prudce klesá.
- **Důsledek pro vývoj:** Ačkoliv model deklaruje podporu stovek tisíc tokenů, při komplexním křížovém refaktoringu ve velkém kontextu často přehlédne klíčové souvislosti.

2.2.8 Promptové inženýrství a negativní instrukce

- **Systémový prompt:** Základní direktiva definující identitu agenta, dostupné nástroje a mantinely (formát commitů, zákaz destruktivních příkazů) (5).
- **Few-shot a Chain-of-Thought (CoT):** Vzorové ukázky řešení a vedení modelu k explicitní formulaci mezikroků uvažování před samotným zápisem kódu.

- **Úskalí negativních instrukcí:** Modely často porušují zákazy formulované negací (např. „nemazat existující testy“), protože matice pozornosti (QK^T) asociativně aktivuje zakázané pojmy dříve, než autoregresní proces uplatní logický operátor negace.
- **Inženýrské řešení:** Afirmativní formulace pravidel (pozitivní vymezení povolených mantinelů) kombinovaná s deterministickou ochranou v harnessu (připojení chráněných souborů pouze pro čtení, blokace v CI).

2.3 Architektura řídicího harnessu a orchestrace

2.3.1 Úvod do řídicích harnessů

- **Vymezení pojmu harness:** Řídicí a dozorčí program obklopující inferenční jádro. Samotné inferenční jádro provádí pouze maticové násobení vah sítě; veškerou orchestraci řídí harness.
- **Funkce harnessu:**
 - Inicializace a správa sezení.
 - Konstrukce a dynamická injekce promptu.
 - Zajištění bezpečného běhu a izolace nástrojů.
 - Řízení stavových přechodů a vynucování bezpečnostních pojistek.

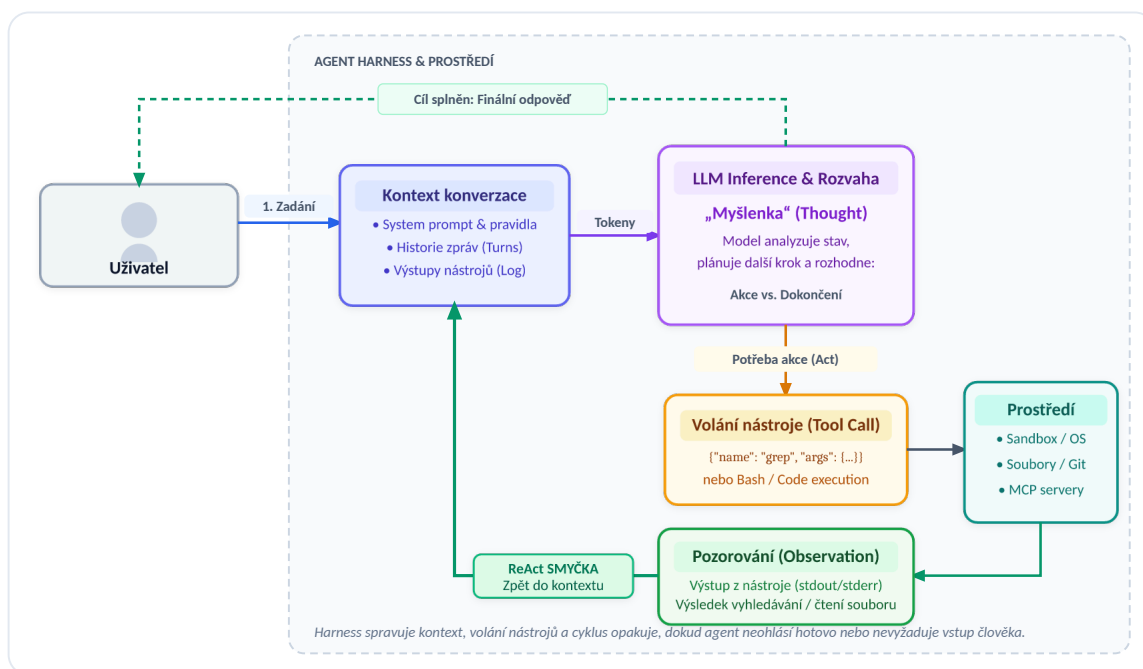
2.3.2 Autonomní agent vs. konverzační chatbot

- **Konverzační chatbot:**
 - Generuje pasivní textové odpovědi v uzavřeném rozhraní.
 - Nemá přímý přístup k souborovému systému ani k operačnímu systému.
 - Uživatel musí navržený kód manuálně zkopírovat, spustit a otestovat.
- **Autonomní agent:**
 - Vybaven sadou výkonných nástrojů (*tools*).
 - Aktivně čte repozitář, modifikuje soubory, spouští testy a interpretuje jejich výstupy.
 - Uzavřen v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu prostředí.

2.3.3 Prováděcí cyklus ReAct

- **ReAct smyčka (*Reasoning + Acting*) (6):** Čtyřfázový prováděcí cyklus znázorněný na Obrázek 1:
 1. **Rozvaha (*Thought*):** Analýza aktuálního stavu kontextu modelem a formulace nejbližšího záměru.
 2. **Volání nástroje (*Tool Call*):** Emitování strukturovaného požadavku na provedení konkrétní akce.
 3. **Vykonání a pozorování (*Observation*):** Bezpečný běh akce v harnessu a vložení výstupu do kontextu.

4. Navazující iterace: Předložení aktualizovaného kontextu modelu v dalším tahu.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.4 Běhové prostředí nástrojů a pískoviště (Sandbox)

- **Strukturované volání nástrojů (Tool / Function Calling):** Validace vstupů a výstupů proti JSON schémátům. Spolehlivé pro atomické operace, ale nese tokenovou režii schémat.
- **Přímé spouštění kódu (Code Execution):** Spouštění generovaných skriptů v sandboxu. Maximální flexibilita, avšak vyžaduje striktní bezpečnostní izolaci.

🔥 **Hlubková kritika / Oponentura: Iluzorní bezpečnost pískoviště:** Přímé spouštění netestovaného syntetického kódu v běžném Docker kontejneru nelze považovat za plnohodnotnou bezpečnostní hranici (*security boundary*). Přístup k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky environmentálních tajností (GitHub tokeny, API klíče k LLM) přes skryté síťové kanály a kompromitaci CI infrastruktury. Pro bezpečný produkční provoz je nezbytná formální izolace na bázi microVM (např. AWS Firecracker, gVisor) a striktní izolace síťových jmenových prostorů.

2.3.5 Patologie divergence: perseverace a oscilace

- **Perseverace a zacyklení:** Opakované emitování identického volání nástroje se stejnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy. V kontextu vzniká pravděpodobnostní atraktor, který nutí model k opakování chybného vzorce.
- **Oscilace a těkání (Thrashing):** Strídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije A).

- **Nekontrolovaná spotřeba zdrojů (*Context Runaway*):** Rychlé vyčerpání kontextového okna i rozpočtu na volání API.

2.3.6 Dovednosti (Skills)

- **Koncept dovedností (*Skills*):** Adresáře instrukcí a referencí se souborem `SKILL.md` (YAML frontmatter).
- **Efektivita kontextu:** Do výchozího systémového promptu se načítají pouze stručná metadata (název a popis role); detailní návod a skripty se načítají dynamicky až při explicitním vyvolání nástroje.
- **Skripty a záchytné body (*Scripts & Hooks*):** Deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

2.3.7 Model Context Protocol (MCP servery)

- **Model Context Protocol (MCP) (7):** Otevřený standard propojující jazykové modely s externími nástroji a datovými zdroji.
- **Protokolové rozhraní:** Komunikace probíhá přes protokol JSON-RPC (prostřednictvím `stdio` nebo `Server-Sent Events / SSE`).
- **Architektonické oddělení:** Striktní oddělení běhového prostředí agenta od implementace nástrojů. MCP servery fungují jako samostatné, znovupoužitelné komponenty běžící mimo jádro harnessu.

2.3.8 Škálování: hierarchičtí subagenti a DAG workflow

- **Subagenti (*Subagents*):** Hierarchická dekompozice úlohy orchestrátorem na specializované agenty (průzkumník repozitáře, plánovač, kódovací dělník). Kontext subagenta je po dokončení zahozen a orchestrátoru je předán pouze čistý výsledek, což chrání primární kontext před znečištěním (*context pollution*).
- **Pracovní postupy jako grafy (DAG / Graph Engineering):** Formalizace fází životního cyklu jako orientovaného acyklického grafu (detekce → plán → kód → testy → schválení). Hrany definují striktní závislosti (`needs`); selhání v libovolném uzlu okamžitě zastaví navazující kroky.

2.3.9 Zapojení člověka do smyčky (Human-in-the-loop)

- **Princip Human-in-the-loop:** Cílem není nekritická plná autonomie, nýbrž automatizace rutinních a mechanických kroků v kombinaci s lidským rozhodováním tam, kde je změna nevratná nebo kde chybí jednoznačné algoritmické měřítko správnosti.
- **Lidské schvalovací brány (*Human Gates*):** Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora.
 - **Dvoufázové schvalování:**

- **1. brána (Záměr a plán):** Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód.
- **2. brána (Sémantická revize):** Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
- ▶ **Prevence únavy z revizí (*Review Fatigue*):** Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- **Dohledatelnost původního zadání:** Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- **Transparentnost selhání a deterministická eskalace:** Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu tahů nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

 **Hlubková kritika / Oponentura: Kognitivní limity lidské schvalovací brány (*Review Fatigue*):** Spoléhání se na finální sémantickou kontrolu diffu v pull requestu naráží na lidské kognitivní limity. Výzkumy prokazují, že u rozsáhlých diffů (nad 300–400 řádků) dramaticky klesá hloubka lidské pozornosti — vývojář kód pouze zběžně prohlédne a spoléhá na zelenou fajfku z CI. Aby byla lidská brána efektivní, harness musí diffy rozkládat do sémanticky sevřených mikrokroků, generovat interaktivní vysvětlení netriviálních rozhodnutí a explicitně zvýrazňovat změny v kritických architektonických komponentách.

3 Praktická část

3.1 Cíl a rozsah systému DarkFactory

- **Systém DarkFactory (8):** Agentní harness instalovatelný jako aplikace pro platformu GitHub (GitHub App).
- **Cíl systému:** Maximalizace automatizace rutinních fází softwarového inženýrství:
 - Sémantická analýza požadavků v GitHub Issues.
 - Technické plánování a dekompozice úlohy.
 - Generování a úprava zdrojového kódu.
 - Běh validačních testů a automatické opravy chyb.
 - Otevření strukturovaného pull requestu.
- **Míra autonomie:** Kooperativní model se zapojením člověka (*Human-in-the-loop*). Vzhledem ke stochastické povaze LLM nelze systém označit za plně autonomní; modely nemohou nést konečnou architektonickou odpovědnost.
- **Dělbá rolí:**
 - **Autonomní agent:** Kognitivně rutinní a mechanické operace (průzkum souborů, syntéza kódu, řešení syntaktických regresí, běh testů).
 - **Lidský inženýr:** Vymezení záměru (co a proč stavět), schvalování technických plánů a finální sémantická kontrola kódu v PR.

 **Hlubková kritika / Oponentura: Bezpečnostní perimetr aplikace GitHub App:** Architektura DarkFactory staví na instalaci jako GitHub App s právy zápisu do repozitáře a správy pull requestů. Pro podnikové nasazení to představuje zásadní vektor útoku: pokud model podlehně nepřímému prompt injection útoku (např. ze zlomyslného komentáře v issue či neznámého balíčku), získává přístup k interním tokenům repozitáře. Při obhajobě je nutné explicitně vymezit, jak harness izoluje tajnosti repozitáře, omezuje síťový provoz sandboxu a vynucuje princip nejmenších oprávnění (*least privilege*).

3.2 Architektura a životní cyklus požadavku

 **Strukturální upozornění: Architektura a životní cyklus v rekonstrukci:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory byla dosavadní implementační dekompozice modulů, stavový automat životního cyklu a konfigurační model dočasně nahrazeny zástupnými strukturálními bloky. Nová architektura, komponentní diagram a specifikace rozhraní budou doplněny po stabilizaci nové verze.

- **Architektonická modularita:** Striktní oddělení deklarativního konfiguračního manifestu klientského repozitáře od sdílené orchestrační logiky.
- **Hermetické běhové prostředí:** Izolovaný běh úloh v kontejnerech se zamčenými verzemi závislostí a nástrojů.

- **Životní cyklus požadavku (Stavový automat / DAG):**
 - ▶ **1. Příjem:** Detekce nového zadání v GitHub Issues.
 - ▶ **2. Plánování:** Analýza kódu a návrh kroků implementace.
 - ▶ **3. Lidská brána 1:** Schválení plánu vývojářem.
 - ▶ **4. Implementace:** Autonomní editace souborů v izolované větvi.
 - ▶ **5. Validace:** Běh testů a kontrol v CI s automatickou samoopravou chyb.
 - ▶ **6. Vystavení PR:** Otevření pull requestu s popisem a diffem.
 - ▶ **7. Lidská brána 2:** Finální kontrola a schválení člověkem.
 - ▶ **8. Integrace:** Automatické sloučení do hlavní větve (*squash and merge*).
- **Dvouúrovňová detekce projektů:**
 - ▶ **Prostředí (Environment):** Určuje vyžadované binární nástroje a balíčkovací manažery podle detekovaných souborů (viz Tabulka 1).
 - ▶ **Doména (Domain):** Určuje způsob řízení výstupů (doména programového kódu vs. textová sazba a dokumentace).

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

- **Deklarativní manifest:** Umožňuje automaticky detekované parametry deklarativně přepsat či rozšířit (např. o explicitní cesty k písmům nebo dodatečné integrační kontroly).

 **Návrh na vylepšení:** Placeholder: Zde bude doplněna nová vrstevnatá dekompozice přestavěného systému DarkFactory, stavový diagram životního cyklu požadavku a deklarativní schéma konfiguračního manifestu.

3.3 Orchestrační jádro a validační infrastruktura

 **Strukturální upozornění: Orchestrace a validační subsystém v rekonstrukci:** Mechanismus rotace poskytovatelů LLM, správa tokenů, verifikační pipeline a správa nastavení jako kód procházejí rekonstrukcí podle nového orchestračního enginu DarkFactory.

* Pojem **hermetické prostředí** označuje výpočetní prostředí zcela izolované od nekontrolovaných stavů hostitelského systému a sítě. Veškeré nástroje, knihovny a závislosti jsou explicitně uzamčeny na konkrétních verzích, což zaručuje determinismus a reprodukovatelnost.

Klíčové operační mechanismy orchestračního jádra:

- **Rotace poskytovatelů LLM:** Úkoly jsou definovány nezávisle na formátu konkrétního poskytovatele. Při vyčerpání kvóty nebo výpadku API primárního modelu harness automaticky přepne na navazující model v rotačním žebříčku, což eliminuje prostoje pipeline.
- **Deterministické ověřování změn:** Každý zásah do kódu podléhá požadovaným kontrolám v CI. Úlohy končí explicitním stavem, čímž brání zablokování větve v důsledku přeskočených kontrol.
- **Správa nastavení jako kód (*Config as Code*):** Pravidla ochrany větví, konfigurace linterů i projektové nástěnky (GitHub Projects) jsou definovány deklarativně a podléhají verzování v gitu.
- **Automaticky odvozovaná živá dokumentace (*Living Documentation*):** Dokumentace a specifikace jsou generovány přímo ze zdrojového kódu a strukturovaných inline komentářů s automatickou validací v CI.

 **Hlubková kritika / Oponentura: Asymetrie formátů a ztráta kontextu při rotaci poskytovatelů:** Rotace modelů různých poskytovatelů (např. přepnutí z Claude na GPT či DeepSeek při vyčerpání kvóty) naráží na zásadní odlišnosti v tokenizérech, syntaxi nástrojů (XML vs. JSON schemas) a vnímání systémového promptu. Při náhlém předání kontextu hrozí ztráta návaznosti v rozpracovaném plánu. Text musí vysvětlit přítomnost normalizační vrstvy (provider adapter), která udržuje historii tahů a výpisy nástrojů v neutrální kanonické reprezentaci.

 **Návrh na vylepšení:** Placeholder: Zde bude doplněn detailní popis nového modelu rotace poskytovatelů, správy tokenových rozpočtů, cachování závislostí a generování živé dokumentace po dokončení přestavby.

3.4 Systém revizních značek pro lidský dohled nad akademickým textem

Protokol vizuálních revizních značek pro formát Typst formalizuje dohled člověka nad generovaným textem:

1. Postranní revizní panely (*Callouty*):

- `#note[...]` (zelený): Konstruktivní náměty na vylepšení, doplnění diagramů a praktické vazby.
- `#issue[...]` (červený): Věcné nesrovnalosti, logické mezery, překlepy a duplicitní obsah.
- `#alert[...]` / `#struct-alert[...]` (žlutý): Strukturální nevyváženost, chybějící sekce a osnovové neshody.
- `#critique[...]` (oranžový): Hlubková oponentura, slepá místa, zpochybnění neověřených předpokladů k obhajobě.

- `#blue-note[...]` (modrý): Metodické vymezení, rozsah práce a konceptuální mantinely zadání.

 **Návrh na vylepšení:** Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

 **Chyba / Nesrovnalost k opravě:** Detekované věcné nepřesnosti, logické mezery, překlapy nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

 **Strukturální upozornění:** Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

 **Hloubková kritika / Oponentura:** Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabin formulovaných jako břitké otázky k obhajobě.

 **Metodické vymezení / Rozsah práce:** Metodické vymezení a rozsah práce — formulace hranic zkoumaného problému, metodická abstrakce (např. oddělení agentního inženýrství od strojového učení) a mantinely zadání.

2. Textové revizní funkce v toku textu:

- **Neověřený koncept** (`#draft[...]` / `#unconfirmed[...]`): Žluté podbarvení — koncept čekající na posouzení a revizi autorem.
- **Nově přidáný text** (`#added[...]`): Zelené podbarvení — nově vygenerovaný text začleněný agentem.
- **Potvrzený text** (`#confirmed[...]`): Modré podbarvení — text potvrzený uživatelem, čekající na finální integraci.
- **Navrženo k odstranění** (`#removed[...]`): Červené přeškrtnutí — zastaralý text navržený ke smazání.
- **Srovnávací diff** (`#diff(old, new)`): Červený původní text přeškrtnutý následovaný zeleným novým textem.
- **Čistý neoznačený text:** Finální, autorsky schválený text v hlase autora bez podbarvení.

 **Návrh na vylepšení: Automatizovaná kontrola revizních značek v CI:** Doporučujeme propojit protokol revizních značek s validační pipeline DarkFactory: CI linter by měl ověřit, že před finálním vydáním nezůstala v rukopisu žádná neuzavřená značka vady (`#issue`) a že publikovaná čistá verze `prace.pdf` deterministicky potlačuje všechny interní callouty a diffy.

4 Výsledky a diskuse

 **Metodické vymezení / Rozsah práce: Metodické vymezení výsledků a diskuse:** V souladu se zaměřením práce na principy agentního inženýrství a architekturu řídicího harnessu se tato kapitola soustředí na systémové a architektonické vlastnosti navrženého řešení: modularitu, determinismus, bezpečnostní izolaci a srovnání s existujícími přístupy (monolitické agentní smyčky vs. řízené DAG workflow).

4.1 Sjedení pracovních postupů a přenositelnost napříč doménami

Architektonické principy centralizovaného harnessu:

- **Centralizace pracovních postupů:** Klientské repozitáře neudrží vlastní izolované skripty; exekuci delegují na centrální znovupoužitelné šablony a specifičnost projektu vymezují v deklarativním manifestu.
- **Údržbová složitost $O(1)$:** Bezpečnostní oprava či aktualizace v harnessu je provedena jednou a spotřebitelské projekty ji přebírají posunem připnuté verze ($O(1)$ oproti $O(N)$ manuálním úpravám v každém repozitáři zvlášť).
- **Statická kontrola jádra:** Jádro harnessu podléhá striktní typové kontrole a jednotkovému testování před každým vydáním.
- **Omezení jediného bodu selhání (SPOF):** Projekty odkazují na neměnný kryptografický SHA hash commitu či sémantický tag, což vylučuje nechtěné regresní změny.
- **Konzistentní agentní prostředí:** Jednotný životní cyklus požadavků, správa kontextu, rotace modelů i vyhodnocení validačních bran napříč všemi projekty.
- **Oddělení prostředí od domény:**
 - ▶ **Doména programového kódu:** Kompilace, statická analýza, jednotkové testy a měření pokrytí.
 - ▶ **Doména textu a dokumentace:** Sazební kompilace, kontrola terminologie a generování auditovatelných PDF artefaktů.
- **Univerzální orchestrační automat:** Řídicí harness obsluhuje obě domény shodným stavovým automatem bez větvení integrační logiky.

 **Hlubková kritika / Oponentura: Riziko kaskádového rozpadu při centralizaci šablon:** Argumentace údržbovou složitostí $O(1)$ zamlčuje druhou stranu mince: chybná nebo nekompatibilní změna v centrálním harnessu okamžitě ohrožuje buildy ve všech klientských repozitářích (kaskádové selhání upstreamu). V textu je nezbytné obhájit, proč je absolutně vyžadováno striktní připínání na konkrétní neměnný SHA hash commitu, a jaké postupy postupného nasazování (canary testing) harness využívá pro ověření změn před jejich plošným nasazením.

4.2 Provozní metriky a spolehlivost agentních běhů

 **Strukturální upozornění: Metriky čekají na novou evaluaci:** Původní ad-hoc měření byla vyřazena. Po dokončení a stabilizaci nové verze harnessu proběhne série strukturovaných měření nad reálnými vývojovými požadavky.

 **Návrh na vylepšení:** Placeholder: Zde bude zařazena nová tabulka empirických metrik (počet zpracovaných požadavků, úspěšnost PR na první pokus, úspěšnost po automatické samoopravě, průměrná spotřeba tokenů na tah, časová latence a spolehlivost rotace modelů).

4.3 Poznatky a systémová úskalí z provozu

Poznatky z nasazení autonomních agentů v CI:

- **Řízení souběžnosti a větvené zámky:** Paralelní integrační běhy vyžadují striktní zámky na úrovni větví, aby nedocházelo ke kolizím a uvážnutí workflow.
- **Transparentnost hlášení selhání:** Zákaz tichého pohlcování výjimek; každá chyba nástroje nebo API musí být zaznamenána do kontextu a eskalována člověku. Tiché maskování chyb vede k masivním halucinacím modelu.
- **Deklarativní detekce prostředí:** Vyloučení implicitních předpokladů o repozitáři; veškeré kroky se deterministicky odvozují z přítomnosti souborů a deklarativního manifestu.

4.4 Diskuse: Porovnání architektur

Porovnání navrženého harnessu se současnými platformami (SWE-agent (6), Devin, Copilot Workspace):

- **Deterministický DAG vs. nekonečná smyčka:**
 - **Volné smyčky (SWE-agent):** Monolitický model v interaktivním terminálu sám rozhoduje o ukončení; při uvíznutí v atraktoru snadno vyčerpá rozpočet tokenů.
 - **Navržený harness:** Model je uzavřen do deterministického grafu (DAG) se striktně alokovaným rozpočtem tahů na každou fázi a izolací kontextu.
- **Granularita lidského dohledu:**
 - **Jednorázový přístup:** Přímé vygenerování celého PR; při špatném pochopení zadání vede ke stovkám řádků vadného kódu a únavě vývojáře z revizí (*review fatigue*).
 - **Dvoufázové schvalování:** Člověk autorizuje záměr a technický plán dříve, než agent začne zasahovat do repozitáře.
- **Bezpečnost a hermetičnost:**
 - **Lokální stroje:** Riziko poškození prostředí či úniku tajemství při spouštění netestovaného kódu.

- ▶ **Efemérní CI kontejnery:** Běh v izolovaném sandboxu s minimálními oprávněními tokenů a auditovatelným protokolem všech operací.

💡 **Návrh na vylepšení: Doporučení empirického srovnání se SWE-bench:** Pro rigorózní akademické zhodnocení doporučujeme po stabilizaci nového jádra DarkFactory podrobit harness standardnímu benchmarku SWE-bench Lite. Porovnání úspěšnosti (`Pass@1`) a spotřeby tokenů vůči referenčním systémům (SWE-agent, AutoCodeRover) poskytne nezpochybnitelný empirický důkaz funkčnosti navržených bezpečnostních pojištění.

4.5 Systémová a metodická omezení

Limity navržené architektury:

- **1. Hranice deterministické verifikovatelnosti:** Autonomie je spolehlivá pouze u objektivně testovatelných změn (kód, testy, typy). U subjektivních úloh (UX ergonomie, grafický design, stylistická formulace textu) zůstává role modelu asistenční a validaci provádí člověk.
- **2. Propustnost a dostupnost inferenčních API:** Rotační žebříček tlumí lokální výpadky, avšak globální kvóty a latence cloudových poskytovatelů tvoří pevný strop průchodnosti pipeline.
- **3. Absence vícevláknového řešení konfliktů:** Souběžná práce mnoha autonomních agentů a lidí nad týmiž soubory a řešení komplexních merge konfliktů představuje otevřenou výzvu pro navazující výzkum.

🔥 **Hlubková kritika / Oponentura: Metodologická závislost na kvalitě testovací sady repozitáře:** Celá premisa deterministického ověřování předpokládá, že repozitář disponuje kvalitní a přísnou testovací sadou. V praxi je však pokrytí kódu testy často nízké nebo testy ověřují pouze triviální scénáře. Pokud agent provede změnu, která projde chudou sadou testů, ale zavleče subtilní architektonickou vadu, deterministická brána selhává. V diskusi je nutné rozebrat pokročilé techniky verifikace, např. mutační testování (*mutation testing*) či generování regresních testů agentem před samotným zásahem do produkčního kódu.

5 Závěr

📌 **Metodické vymezení / Rozsah práce: Poznámka k vypracování závěru:** Závěr práce bude sepsán jako poslední krok po definitivním ucelení teoretických východisek agentního inženýrství a detailní architektury řídicího harnessu. Tato závěrečná kapitola syntetizuje zjištění o deterministickém řízení autonomních agentů a zhodnotí formulované principy a výzkumné otázky bez vazby na dřívější ad-hoc testovací repozitáře.

KONCEPT

Seznam příloh

A	Obsah přiloženého média	27
B	Schéma konfiguračního manifestu darkfactory.json	28
C	Sdílené workflow pro GitHub Actions	29
D	Systémové prompty plánovacího a kódovacího agenta	30
E	Protokol revizních značek v sazebním systému Typst	31

KONCEPT

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib               # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/              # Sdílené šablony kontinuální
integrace
│   └─ src/                             # Výkonné orchestrační jádro systému
├─ out/prace.pdf                        # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

 **Strukturální upozornění: Příloha v rekonstrukci:** Formální JSON schéma manifestu bude doplněno po stabilizaci nového schématu v přestavěném systému DarkFactory.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedeno úplné JSON Schema vymezující validní syntaxi nového konfiguračního manifestu.

KONCEPT

C Sdílené workflow pro GitHub Actions

 **Strukturální upozornění: Příloha v rekonstrukci:** Definice sdílených a volajících workflow pro GitHub Actions budou aktualizována po dokončení přestavby orchestrátoru.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny ukázky znovupoužitelného a volajícího workflow nové verze DarkFactory.

KONCEPT

D Systémové prompty plánovacího a kódovacího agenta

 **Strukturální upozornění: Příloha v rekonstrukci:** Systémové prompty pro plánovací a implementační fáze budou aktualizovány podle nových rolí a instrukčních šablon v přestavěném systému.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny kompletní systémové prompty pro jednotlivé fáze životního cyklu požadavku.

KONCEPT

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatel zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břitká, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabin metodiky a příprava na otázky zkušební komise.

 **Metodické vymezení / Rozsah práce:** Ukázka modrého panelu metodického vymezení (`#blue-note` / `#scope-note`): Vymezení rozsahu práce, formulace mantinelů a architektonických abstrakcí.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** (`#diff`): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text:** Představuje finální, autorsky schválený text v hlase autora bez jakéhokoli podbarvení.

Role značek v lidském dohledu:

- **Čistý neoznačený text:** Schválený autorský text v hlase autora.
- **Žluté podbarvení** (`#unconfirmed`): Neověřený koncept čekající na lidské posouzení.
- **Zelené podbarvení** (`#added`): Nově vygenerované návrhy agenta.
- **Modré podbarvení** (`#confirmed`): Uživatelem odsouhlasený text.
- **Srovnávací diff** (`#diff`): Transparentní vizualizace navržených oprav.
- **Postranní panely (Callouty):** Striktní oddělení námětů, chyb a oponentury od těla textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
5. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
6. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
7. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
8. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
9. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>